

GPUMODE Lecture Study Notes: Lecture 9 Reductions

Contents

1 Core Problem the Lecture Addresses

The lecture studies **parallel reductions** on GPUs. A reduction is an operation that takes a larger input collection and collapses it into a smaller output, often a single scalar. In machine learning systems, reductions appear constantly: sums, products, minimums, maximums, means, norms, argmax operations, loss aggregation, pooling, and the normalization denominator inside softmax.

The core technical problem is:

How do we map a logically serial operation, such as summing all values in a vector, onto thousands of GPU threads while preserving correctness, minimizing divergence, using the memory hierarchy efficiently, and handling large inputs?

A serial reduction has a dependency chain. For a sum over N elements,

$$s = \sum_{i=0}^{N-1} x_i,$$

one naive CPU-style implementation updates one accumulator repeatedly:

$$s_{i+1} = s_i + x_i.$$

This is simple, but it exposes little parallelism because each update depends on the previous update. A GPU implementation must break this chain into many independent partial reductions and then combine those partial results.

The lecture develops the reduction kernel step by step:

1. Start with the mathematical abstraction of reductions.
2. Show a serial reduction and why it is not GPU-friendly.
3. Introduce the **parallel reduction tree**.
4. Implement a naive CUDA reduction using pairwise operations.
5. Improve control divergence by changing the thread-to-data mapping.
6. Use shared memory to avoid repeated global memory traffic.
7. Extend the algorithm to large arrays with segmented multi-block reductions.
8. Use **thread coarsening** so each thread reduces multiple elements.
9. Connect these ideas to PyTorch, TorchInductor, Triton, and generic reduction kernels.

The main engineering lesson is that reductions are not hard because the math is complicated. They are hard because the **thread strategy**, **memory strategy**, **synchronization strategy**, and **numerical strategy** all matter.

2 Required Background

2.1 What Is a Reduction?

A reduction combines elements of a collection using a binary operation. If the input is a vector

$$\mathbf{x} = [x_0, x_1, \dots, x_{N-1}],$$

then a reduction computes

$$r = (((e \text{ op } x_0) \text{ op } x_1) \cdots \text{op } x_{N-1}),$$

where e is the **identity element** for the operation.

Examples include:

Reduction	Operation	Identity
sum	$a + b$	0
product	$a \cdot b$	1
maximum	$\max(a, b)$	$-\infty$
minimum	$\min(a, b)$	$+\infty$
logical and	$a \wedge b$	true
logical or	$a \vee b$	false

The identity element satisfies

$$e \text{ op } x = x.$$

For example,

$$0 + x = x,$$

and

$$1 \cdot x = x.$$

2.2 Reduction as a Generic Programming Pattern

Many reductions share the same control structure. In pseudocode:

```
def reduce(data, identity, op):  
    result = identity  
    for x in data:  
        result = op(result, x)  
    return result
```

This abstraction is important because production frameworks do not want to maintain separate hand-written kernels for every possible reduction. Instead, they often use a general reduction infrastructure parameterized by:

- the input type;
- the output type;
- the accumulator type;
- the identity value;
- the reduction operator;
- the shape and stride metadata;
- the chosen launch configuration;
- heuristics for block size and values per thread.

For example, a framework may have one generic reduction kernel template that can be instantiated for sum, max, min, product, mean, or norm.

2.3 Where Reductions Appear in Machine Learning

Reductions are ubiquitous in deep learning. Important examples include:

- **Mean pooling:** average values over a spatial tile.
- **Max pooling:** take the maximum over a spatial tile.
- **Classification:** take an argmax over class probabilities or logits.
- **Loss functions:** reduce per-example losses into a scalar objective.
- **Softmax normalization:** compute a denominator by summing exponentials.
- **Layer normalization:** compute mean and variance over a feature dimension.
- **Batch normalization:** compute statistics over batch and spatial dimensions.
- **Attention:** compute row-wise max and row-wise sums in numerically stable softmax.
- **Gradient aggregation:** sum gradients across examples, devices, or tensor axes.

Softmax itself maps a vector to a vector, so the full operation is not a scalar reduction. However, the denominator is a reduction:

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=0}^{N-1} \exp(x_j)}.$$

For numerical stability, softmax is commonly written as

$$\text{softmax}(x_i) = \frac{\exp(x_i - m)}{\sum_{j=0}^{N-1} \exp(x_j - m)},$$

where

$$m = \max_{0 \leq j < N} x_j.$$

This contains two reductions:

$$m = \max_j x_j,$$

and

$$d = \sum_j \exp(x_j - m).$$

2.4 CUDA Execution Model Reminder

A CUDA kernel is launched over a grid of thread blocks. Each block contains threads. Threads are grouped into warps, usually 32 threads per warp on NVIDIA GPUs.

Important CUDA identifiers are:

`threadIdx.x`, `blockIdx.x`, `blockDim.x`, `gridDim.x`.

A common global linear index is

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

For reductions, this simple one-thread-per-output strategy does not directly apply because there may be only one output scalar. We therefore need many threads to cooperate on one output.

3 Main Concepts

3.1 Serial Reduction

For an example vector

$$\mathbf{x} = [5, 2, 8, 1],$$

a serial maximum reduction starts with

$$m_0 = -\infty.$$

Then it updates:

$$m_1 = \max(-\infty, 5) = 5,$$

$$m_2 = \max(5, 2) = 5,$$

$$m_3 = \max(5, 8) = 8,$$

$$m_4 = \max(8, 1) = 8.$$

The final result is

$$\max(\mathbf{x}) = 8.$$

This is a dependency chain. The value m_3 depends on m_2 , which depends on m_1 , which depends on m_0 . A single thread can perform this easily, but a GPU would be underutilized.

3.2 Parallel Reduction Tree

The key algorithmic idea is to reduce pairs of elements in parallel. For a sum reduction over 8 values,

$$[x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7],$$

the first layer computes:

$$y_0 = x_0 + x_1,$$

$$y_1 = x_2 + x_3,$$

$$y_2 = x_4 + x_5,$$

$$y_3 = x_6 + x_7.$$

The next layer computes:

$$z_0 = y_0 + y_1,$$

$$z_1 = y_2 + y_3.$$

The final layer computes:

$$r = z_0 + z_1.$$

The depth of this tree is

$$\log_2 N$$

when N is a power of two. The total number of binary operations is still

$$N - 1,$$

but the parallel span is much shorter. Instead of N serial steps, the algorithm requires approximately

$$\lceil \log_2 N \rceil$$

synchronized stages.

Visual Concept

Draw a binary tree. The leaves are input elements. Each internal node combines two children. The root is the final reduced scalar. Label each level as one parallel reduction step.

3.3 Associativity and Floating-Point Non-Determinism

Mathematically, addition is associative:

$$(a + b) + c = a + (b + c).$$

Floating-point addition is not exactly associative because numbers are rounded to a finite representation after each operation. In floating-point arithmetic, it is possible that

$$\text{fl}(\text{fl}(a + b) + c) \neq \text{fl}(a + \text{fl}(b + c)).$$

This matters because parallel reductions change the order of operations. A left-to-right serial sum and a tree-structured parallel sum may produce slightly different answers.

The lecture gives an example with many tiny values and large positive and negative values. Suppose the input is conceptually similar to

$$[10^{-20}, 10^{-20}, \dots, 10^{-20}, 1, -1].$$

If tiny values are added to a large value too early, they may vanish due to rounding. If tiny values are accumulated together first, they may survive longer. Therefore, reduction order affects numerical accuracy.

This is one reason GPU results can appear non-deterministic. Another reason is that thread scheduling and atomic operation ordering may not be fully deterministic. In PyTorch, users can request deterministic behavior with deterministic algorithm settings, but determinism is not free. It can force stricter synchronization or alternative algorithms that are slower.

3.4 Accumulator Precision

For low-precision data types such as FP16 or BF16, reductions are often accumulated in FP32. If the input values are x_i , where x_i is stored in FP16, the accumulator may be FP32:

$$s_{\text{fp32}} = \sum_{i=0}^{N-1} \text{cast}_{\text{fp32}}(x_i).$$

Only after the accumulation is complete might the result be converted back to FP16:

$$y_{\text{fp16}} = \text{cast}_{\text{fp16}}(s_{\text{fp32}}).$$

This pattern appears in GEMM and reductions. In Triton matrix multiplication tutorials, a common pattern is to multiply lower-precision inputs but keep the accumulator in FP32:

```
accumulator = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)

for k in range(0, K, BLOCK_K):
    a = tl.load(a_ptrs)
    b = tl.load(b_ptrs)
    accumulator += tl.dot(a, b)

output = accumulator.to(tl.float16)
tl.store(c_ptrs, output)
```

The reason is that accumulation is sensitive to rounding. A reduction over many low-precision values can lose information if the accumulator is too narrow.

4 Hardware-Level Explanation

4.1 Why One Thread Is Not Enough

If one CUDA thread performs the entire reduction, the kernel has almost no parallelism. For an input of size N , one thread performs $N - 1$ operations while the rest of the GPU is idle.

The GPU wants many active warps per SM so it can hide latency. If a reduction uses only one active thread, then:

- global memory latency cannot be hidden effectively;
- CUDA cores are underutilized;
- warp schedulers have little useful work to issue;
- occupancy may look irrelevant because the work itself is serialized;
- the kernel launch overhead may dominate for small inputs.

4.2 Thread Blocks and Shared Memory Cooperation

Within one CUDA block, threads can cooperate using shared memory and barriers. Shared memory is explicitly managed memory located close to the SM. It is much faster than global memory when used properly.

A typical block-level reduction does:

1. Load values from global memory.
2. Place partial values in shared memory.
3. Synchronize with `__syncthreads()`.
4. Repeatedly halve the number of active threads.
5. Have thread 0 write the block's partial result.

The synchronization is needed because one thread may read a value written by another thread in a previous reduction step. Without a barrier, a thread could read stale or uninitialized shared memory.

4.3 Global Memory Versus Shared Memory

Global memory has high latency and high bandwidth. Shared memory has much lower latency and is designed for cooperation inside a block.

A reduction that repeatedly reads and writes intermediate values in global memory is inefficient because intermediate values travel through the memory hierarchy unnecessarily.

The preferred strategy is:

global memory $\rightarrow$ shared memory $\rightarrow$ registers and shared memory $\rightarrow$ global output.

The first load from global memory is necessary. After that, most intermediate reductions should happen in shared memory or registers.

4.4 Control Divergence

Warp divergence occurs when threads in the same warp follow different control paths. In reduction kernels, divergence often happens because fewer and fewer threads are active at later stages.

For example, if a block starts with 1024 threads, then a tree reduction uses:

$$512, 256, 128, 64, 32, 16, 8, 4, 2, 1$$

active logical operations across successive stages.

At later stages, most threads are inactive. This is inherent to the tree shape, but the implementation can make it better or worse.

A bad mapping may scatter active threads across warps, causing many warps to be partially active. A better mapping keeps active threads contiguous so whole warps remain active for longer and inactive work is localized.

4.5 Memory Coalescing

Memory coalescing means that threads in a warp access contiguous memory addresses, allowing the hardware to combine memory transactions efficiently.

For a warp of 32 threads, a coalesced pattern is:

$$\text{thread } t \rightarrow x[i + t].$$

An uncoalesced or strided pattern is:

$$\text{thread } t \rightarrow x[i + s \cdot t],$$

where s is a large stride.

Reduction algorithms can accidentally create poor memory behavior if active threads access locations far apart. The lecture emphasizes changing the thread strategy so active threads are contiguous and access nearby memory.

4.6 Atomics Across Blocks

Threads in the same block can synchronize with `__syncthreads()`. Blocks in the same kernel launch cannot generally synchronize with each other. Therefore, when multiple blocks produce partial sums and all need to update one output scalar, a common simple approach is to use an atomic operation:

$$\text{atomicAdd}(\text{output}, \text{partial_sum}).$$

An atomic operation ensures that the update is read-modify-written safely. Without atomics, two blocks could race:

block A reads y , block B reads y , both write $y + \text{partial}$.

One update could be lost.

Atomics preserve correctness, but they may become a bottleneck if too many blocks contend for one memory location. A more scalable production approach may use a two-pass reduction:

1. First kernel: each block writes one partial result to an intermediate array.
2. Second kernel: reduce the intermediate array.

5 Mathematical Reasoning

5.1 Work and Span

A serial reduction over N elements performs $N - 1$ binary operations and has span

$$T_{\infty}^{\text{serial}} = N - 1.$$

A balanced tree reduction also performs $N - 1$ binary operations, but the span is

$$T_{\infty}^{\text{tree}} = \lceil \log_2 N \rceil.$$

The total work is not reduced. The critical path is reduced. This is the essence of parallel reduction.

5.2 Ideal Parallel Time

If the GPU could execute all operations at each tree level perfectly in parallel, the time would scale like

$$T(N) \approx \alpha \log_2 N,$$

where α is the time per synchronized reduction stage.

In reality,

$$T(N) = T_{\text{load}} + T_{\text{shared}} + T_{\text{sync}} + T_{\text{store}} + T_{\text{launch}},$$

where:

- T_{load} is time to load from global memory;
- T_{shared} is time for shared-memory reductions;
- T_{sync} is synchronization overhead;

- T_{store} is time to write partial or final outputs;
- T_{launch} is kernel launch overhead.

For small N , T_{launch} may dominate. This is why tiny reductions may not benefit much from custom GPU kernels.

5.3 Arithmetic Intensity

Arithmetic intensity is

$$\text{AI} = \frac{\text{FLOPs}}{\text{bytes moved}}.$$

For a simple FP32 sum reduction over N elements, the work is approximately

$$N - 1$$

floating-point additions. The minimum global memory traffic is approximately

$$4N + 4$$

bytes: $4N$ bytes to read the input and 4 bytes to write the final scalar. Therefore,

$$\text{AI} \approx \frac{N - 1}{4N + 4}.$$

For large N ,

$$\text{AI} \rightarrow \frac{1}{4}.$$

This is very low. A simple sum reduction is usually **memory-bandwidth-bound**, not compute-bound. The GPU spends most of its time moving data rather than performing arithmetic.

5.4 Reduction Tree Indexing

A common in-place tree reduction uses a stride. If the stride starts at 1, then the first step combines adjacent pairs:

$$x[0] \leftarrow x[0] + x[1],$$

$$x[2] \leftarrow x[2] + x[3],$$

$$x[4] \leftarrow x[4] + x[5].$$

At step k , the stride is

$$s_k = 2^k.$$

A thread may compute

$$i = 2s_k \cdot t,$$

then update

$$x[i] \leftarrow x[i] + x[i + s_k].$$

This method is conceptually simple but can produce poor divergence because active thread indices become sparse.

An improved version starts with a large stride and halves it:

$$s_0 = \frac{\text{blockDim.x}}{2},$$

$$s_{k+1} = \frac{s_k}{2}.$$

At each step, active threads satisfy

$$t < s_k,$$

and compute

$$x[t] \leftarrow x[t] + x[t + s_k].$$

This keeps active threads contiguous:

$$t = 0, 1, 2, \dots, s_k - 1.$$

That is better for warp execution and memory locality.

5.5 Segmented Multi-Block Reduction

For large arrays, one block cannot reduce the entire input. Let each block process B elements. If the input length is N , the number of blocks is approximately

$$G = \left\lceil \frac{N}{B} \right\rceil.$$

Each block computes a partial sum

$$p_b = \sum_{i=bB}^{\min((b+1)B, N)-1} x_i.$$

The final result is

$$r = \sum_{b=0}^{G-1} p_b.$$

This is a reduction over reductions. The hierarchy is:

within thread $\rightarrow$ within block $\rightarrow$ across blocks.

5.6 Thread Coarsening

Thread coarsening means that each thread processes multiple elements before participating in the block-level reduction.

If each thread processes C values, then thread t may compute a local sum:

$$\ell_t = \sum_{j=0}^{C-1} x[\text{base} + t + j \cdot \text{blockDim.x}].$$

Then the block reduces the local sums:

$$p_b = \sum_t \ell_t.$$

Thread coarsening can improve performance because it:

- reduces the number of active threads needed for the same input size;
- increases per-thread work;
- amortizes indexing overhead;
- may improve memory throughput by having each thread issue multiple loads;
- can reduce the number of blocks and atomic updates.

However, excessive coarsening can hurt occupancy by increasing register pressure or reducing available parallelism.

6 Code Walkthrough

6.1 Reference CPU Reduction

A CPU reference is useful for correctness testing. For example, if the input has 2048 elements and every element is 1.0, then the expected sum is

2048.

```
float cpu_sum(const float* input, int n) {
    float result = 0.0f;

    for (int i = 0; i < n; ++i) {
        result += input[i];
    }

    return result;
}
```

This is serial. It is valuable as a correctness oracle but not as the target GPU algorithm.

6.2 Naive Pairwise CUDA Reduction

A simple parallel reduction assigns one thread to each pair of input elements. The first implementation starts with adjacent pairs, then increases the stride.

```
__global__ void simple_reduce(float* input, float* output) {
    unsigned int t = threadIdx.x;
    unsigned int i = 2 * t;

    for (unsigned int stride = 1;
         stride <= blockDim.x;
         stride *= 2) {

        if (t % stride == 0) {
            input[i] += input[i + stride];
        }

        __syncthreads();
    }

    if (t == 0) {
        *output = input[0];
    }
}
```

6.2.1 Line-by-Line Explanation

The thread index is

```
t = threadIdx.x.
```

The data index is

$$i = 2t.$$

This means thread 0 starts at element 0, thread 1 starts at element 2, and so on. Initially, each thread combines two adjacent elements.

The stride begins as

$$\text{stride} = 1.$$

At each stage, the stride doubles:

$$1, 2, 4, 8, \dots$$

The update

```
input[i] += input[i + stride]
```

combines the current element with another element located `stride` positions away.

The barrier

```
__syncthreads()
```

is required because later iterations read values produced by earlier iterations.

6.2.2 Why This Is Inefficient

The condition

```
if (t % stride == 0)
```

causes divergence. Threads inside a warp may disagree about whether they should execute the body. As the stride changes, the active threads become sparse.

This creates poor branch efficiency because many threads are launched but do no useful work at later stages.

Visual Concept

Draw a row of elements. At stride 1, show thread 0 combining positions 0 and 1, thread 1 combining positions 2 and 3, and so on. At stride 2, show only some threads active. Highlight inactive threads in gray.

6.3 Improved Reduction with Contiguous Active Threads

The next version changes the thread strategy. Instead of starting with adjacent pairs and increasing the stride, it starts with a large stride and halves it.

```
__global__ void reduce_less_divergence(float* input, float* output) {
    unsigned int t = threadIdx.x;

    for (unsigned int stride = blockDim.x;
         stride > 0;
         stride /= 2) {

        if (t < stride) {
            input[t] += input[t + stride];
        }

        __syncthreads();
    }

    if (t == 0) {
        *output = input[0];
    }
}
```

6.3.1 Indexing Logic

At the first stage, if

$$\text{blockDim.x} = 1024,$$

then

$$\text{stride} = 1024.$$

Thread t combines

$$x[t]$$

with

$$x[t + 1024].$$

At the next stage,

$$\text{stride} = 512,$$

so thread t combines

$$x[t]$$

with

$$x[t + 512].$$

The active threads are always contiguous:

$$t < \text{stride}.$$

This is better than using a modulo condition because contiguous threads map more naturally to contiguous warps.

6.3.2 Hardware Behavior

This version improves:

- **branch efficiency**, because active threads are grouped together;
- **warp utilization**, because early stages keep whole warps active;
- **memory locality**, because active threads access nearby locations;
- **cache behavior**, because the access pattern is more regular.

The lecture reports that branch efficiency improved dramatically when switching from the naive strategy to this contiguous-thread strategy.

6.4 Shared Memory Block Reduction

The next improvement is to avoid repeated global memory accesses. Each block loads values into shared memory and then performs the reduction there.

```
__global__ void shared_reduce(const float* input, float* output) {
    __shared__ float input_s[1024];

    unsigned int t = threadIdx.x;
    unsigned int i = t;

    input_s[t] = input[i] + input[i + blockDim.x];

    __syncthreads();

    for (unsigned int stride = blockDim.x / 2;
         stride > 0;
         stride /= 2) {
```

```

        if (t < stride) {
            input_s[t] += input_s[t + stride];
        }

        __syncthreads();
    }

    if (t == 0) {
        *output = input_s[0];
    }
}

```

6.4.1 What the Shared Memory Array Does

The declaration

```
__shared__ float input_s[1024]
```

creates a shared memory buffer visible to all threads in the block. Each block has its own instance of this shared memory.

The line

```
input_s[t] = input[i] + input[i + blockDim.x]
```

performs the first reduction while loading from global memory. If there are 1024 threads, this line reduces 2048 global values into 1024 shared values.

After that, the rest of the reduction happens in shared memory:

```
input_s[t] += input_s[t + stride].
```

6.4.2 Why This Can Be Faster

Global memory is expensive. Shared memory is much closer to the SM. The improved memory path is:

global loads → shared memory partials → shared memory tree reduction → one global store.

This reduces global memory traffic for intermediate states.

6.4.3 Important Limitation

This kernel assumes that the input size fits the block-level reduction pattern. If the input has more elements than the kernel expects, it will not correctly reduce the entire array.

For example, if the kernel is written for roughly 2048 elements but the input has 10000 elements, then many input values are ignored. The result becomes incorrect.

This motivates segmented multi-block reduction.

6.5 Segmented Multi-Block Reduction

For large inputs, launch multiple blocks. Each block reduces one segment of the input, then contributes its partial result to the final output.

```
__global__ void segmented_reduce(const float* input,
                                float* output,
                                int n) {
    __shared__ float input_s[1024];

    unsigned int t = threadIdx.x;
    unsigned int global_i =
        blockIdx.x * blockDim.x * 2 + threadIdx.x;

    float local = 0.0f;

    if (global_i < n) {
        local += input[global_i];
    }

    if (global_i + blockDim.x < n) {
        local += input[global_i + blockDim.x];
    }

    input_s[t] = local;

    __syncthreads();

    for (unsigned int stride = blockDim.x / 2;
         stride > 0;
         stride /= 2) {

        if (t < stride) {
            input_s[t] += input_s[t + stride];
        }

        __syncthreads();
    }

    if (t == 0) {
        atomicAdd(output, input_s[0]);
    }
}
```

6.5.1 Global and Local Indices

The local index is

$$t = \text{threadIdx.x}.$$

The global index is

$$\text{global_i} = \text{blockIdx.x} \cdot \text{blockDim.x} \cdot 2 + \text{threadIdx.x}.$$

The factor of 2 appears because each thread initially loads up to two elements:

$$x[\text{global_i}] \quad \text{and} \quad x[\text{global_i} + \text{blockDim.x}].$$

This means each block processes up to

$$2 \cdot \text{blockDim.x}$$

elements.

6.5.2 Boundary Checks

The checks

```
if (global_i < n) {
    local += input[global_i];
}

if (global_i + blockDim.x < n) {
    local += input[global_i + blockDim.x];
}
```

are necessary when N is not exactly divisible by the block processing size. Without these guards, the kernel could read out of bounds.

6.5.3 Atomic Add

At the end, thread 0 from each block executes

$$\text{atomicAdd}(\text{output}, \text{input_s}[0]).$$

This is required because all blocks update the same output scalar. The atomic operation prevents lost updates.

However, atomics serialize updates to the same memory location. If many blocks call `atomicAdd` on the same output, this final accumulation can become a bottleneck.

6.6 Thread Coarsened Reduction

Thread coarsening gives each thread more work. Instead of loading one or two elements, each thread accumulates several values locally in a register.

```
template <int COARSENING_FACTOR>
__global__ void coarsened_reduce(const float* input,
                                float* output,
                                int n) {
    __shared__ float input_s[1024];

    unsigned int t = threadIdx.x;
    unsigned int block_start =
        blockIdx.x * blockDim.x * COARSENING_FACTOR;

    float local_sum = 0.0f;

    for (int c = 0; c < COARSENING_FACTOR; ++c) {
        unsigned int idx = block_start + c * blockDim.x + t;

        if (idx < n) {
            local_sum += input[idx];
        }
    }

    input_s[t] = local_sum;

    __syncthreads();

    for (unsigned int stride = blockDim.x / 2;
         stride > 0;
         stride /= 2) {

        if (t < stride) {
            input_s[t] += input_s[t + stride];
        }

        __syncthreads();
    }

    if (t == 0) {
        atomicAdd(output, input_s[0]);
    }
}
```

6.6.1 Thread-Level Reduction

The loop

```
for (int c = 0; c < COARSENING_FACTOR; ++c) {
    unsigned int idx = block_start + c * blockDim.x + t;

    if (idx < n) {
```

```

        local_sum += input[idx];
    }
}

```

performs a reduction inside a single thread. The local accumulator is stored in a register:

$$\ell_t = \sum_{c=0}^{C-1} x[\text{block_start} + c \cdot \text{blockDim.x} + t].$$

Then the block reduces all ℓ_t values.

6.6.2 Three-Level Hierarchy

This kernel has three reduction levels:

thread-level reduction $\rightarrow$ block-level shared-memory reduction $\rightarrow$ grid-level atomic reduction.

The lecture emphasizes that the thread strategy is not one uniform rule. The best strategy changes depending on the hierarchy level:

- inside a thread, use registers and sequential accumulation;
- inside a block, use shared memory and synchronization;
- across blocks, use atomics or multiple kernel passes.

6.7 Host-Side Launch Example

A simple driver allocates input, fills it with ones, launches a kernel, and checks that the result equals N .

```

int main() {
    int n = 10000;
    int threads = 1024;
    int values_per_block = threads * 2;
    int blocks = (n + values_per_block - 1) / values_per_block;

    float* h_input = new float[n];

    for (int i = 0; i < n; ++i) {
        h_input[i] = 1.0f;
    }

    float* d_input = nullptr;
    float* d_output = nullptr;

    cudaMalloc(&d_input, n * sizeof(float));

```

```
    cudaMalloc(&d_output, sizeof(float));

    cudaMemcpy(d_input,
               h_input,
               n * sizeof(float),
               cudaMemcpyHostToDevice);

    cudaMemset(d_output, 0, sizeof(float));

    segmented_reduce<<<blocks, threads>>>(d_input, d_output, n);

    float h_output = 0.0f;

    cudaMemcpy(&h_output,
               d_output,
               sizeof(float),
               cudaMemcpyDeviceToHost);

    printf("sum = %f\n", h_output);

    cudaFree(d_input);
    cudaFree(d_output);
    delete[] h_input;

    return 0;
}
```

6.7.1 Correctness Test

If every input element is 1.0, then the correct result is

$$\sum_{i=0}^{N-1} 1 = N.$$

This makes correctness easy to check. For $N = 10000$, the output should be

$$10000.$$

6.8 Profiling with Nsight Compute

The lecture recommends profiling kernels rather than guessing. Useful metrics include:

- **branch efficiency**: measures how much divergence occurs;
- **L1 hit rate**: indicates whether accesses benefit from cache locality;
- **global memory throughput**: shows whether the kernel is bandwidth-bound;
- **shared memory throughput**: shows whether shared memory is heavily used;

- **achieved occupancy:** shows how many warps are active relative to the hardware limit;
- **kernel duration:** the measured GPU kernel time.

The lecture specifically notes that using external timers can be misleading for very small kernels because kernel launch overhead can dominate. Nsight Compute kernel duration is a better way to inspect the actual device-side work.

7 Performance Intuition

7.1 Why the Naive Pairwise Reduction Is Slow

The naive pairwise version has several problems:

1. Active threads become sparse.
2. Warps contain a mix of active and inactive lanes.
3. The modulo condition creates branch divergence.
4. The memory access pattern is less friendly.
5. Many threads are launched but do no work at later stages.

The hardware executes at warp granularity. If only one lane in a warp is active, the warp still consumes scheduling resources. Therefore, keeping active lanes contiguous is important.

7.2 Why Reversing the Stride Helps

The improved algorithm starts with a large stride and halves it. The active condition is

$$t < \text{stride}.$$

This means active threads are

$$0, 1, 2, \dots, \text{stride} - 1.$$

That is much better than a condition such as

$$t \bmod \text{stride} = 0,$$

which activates sparse threads. Contiguous active threads preserve warp efficiency for more stages.

7.3 Why Shared Memory Helps

Shared memory helps because the reduction creates intermediate values. If each intermediate value is written to and read from global memory, the kernel pays unnecessary global memory latency and bandwidth cost.

The shared-memory version does only the necessary global memory work:

```
read input once  and  write output once per block.
```

Intermediate tree operations happen in shared memory.

7.4 Why Shared Memory Alone Is Not Enough

A shared-memory block has finite capacity. A kernel that assumes the whole input fits into one block will fail for large inputs. For example, if a block handles 2048 values but the input has 10000 values, then the kernel must use multiple blocks.

This is the transition from a **single-block reduction** to a **segmented multi-block reduction**.

7.5 Why Atomics Are Correct but Potentially Slow

Atomics solve correctness when many blocks update one scalar. But atomics can serialize the final update. If G blocks all execute

```
atomicAdd(output, partial)
```

on the same address, the memory system must serialize those updates enough to preserve correctness.

For very large reductions, a two-pass algorithm can be better:

1. Each block writes one partial sum to a global array.
2. A second kernel reduces the partial sums.

This reduces atomic contention and may improve scalability.

7.6 Why Thread Coarsening Helps

Thread coarsening increases useful work per thread. A single thread accumulates multiple values in registers before writing to shared memory. This can improve performance because registers are extremely fast and because the kernel does less synchronization relative to the amount of data processed.

However, coarsening is a tuning knob. If the coarsening factor is too high, it may:

- increase register pressure;

- reduce occupancy;
- reduce memory-level parallelism;
- make each thread too heavy;
- worsen load imbalance near boundaries.

The optimal coarsening factor depends on the input size, data type, GPU architecture, memory bandwidth, and block size.

7.7 Small Inputs Versus Large Inputs

For small inputs, a custom GPU reduction may not be worth it. Kernel launch overhead can dominate.

For large inputs, memory bandwidth and reduction hierarchy dominate. The algorithm should:

- load global memory coalescently;
- reduce in registers and shared memory;
- minimize synchronization;
- avoid excessive atomics;
- use multiple blocks;
- choose a good values-per-thread heuristic.

7.8 Bandwidth-Bound Nature of Reductions

A simple sum reduction performs roughly one floating-point operation per input element. For FP32, each element costs 4 bytes to load. The arithmetic intensity approaches

$$\frac{1}{4}$$

FLOPs per byte. This is far below the arithmetic intensity needed to saturate modern GPU compute units. Therefore, reduction performance is often limited by memory bandwidth, not by raw FLOPs.

8 Framework-Level Reduction Kernels

8.1 Why Frameworks Use Generic Reduction Infrastructure

PyTorch exposes user-facing operations such as:

```
torch.sum(x)
torch.max(x)
torch.min(x)
torch.mean(x)
torch.argmax(x)
torch.linalg.norm(x)
```

But internally, frameworks do not want a completely separate hand-written CUDA kernel for every operation, data type, shape, and layout. Instead, they build generic reduction infrastructure.

A generic reduction kernel can be parameterized by:

- the operation, such as sum or max;
- the identity value;
- input and output dtypes;
- accumulator dtype;
- tensor shape and strides;
- whether the reduction dimension is contiguous;
- how many values each thread should process;
- block width and block height;
- whether vectorized loads are legal;
- whether the output is a scalar or tensor.

8.2 Why Code Generation Matters

If a framework had to ship one binary kernel for every combination of operation, dtype, shape class, and layout, the binary size would explode. Code generation and templates avoid this.

The framework can generate or instantiate specialized kernels based on runtime or compile-time information.

The general idea is:

high-level op → reduction description → kernel configuration → generated CUDA or Triton kernel.

8.3 Accumulator Type Heuristics

A framework must decide whether accumulation should happen in the same dtype as the input or a wider dtype.

For example, for FP16 input:

$$x_i \in \text{FP16},$$

a good accumulator is often:

$$s \in \text{FP32}.$$

For integer reductions, the rules differ. For max or min, widening may not be necessary in the same way. For sum or mean, widening can reduce overflow or precision loss.

A robust framework must reason about:

- input dtype;
- output dtype;
- operation type;
- whether accumulation is numerically sensitive;
- whether deterministic behavior is requested.

8.4 Torch Compile and Triton Reductions

When using `torch.compile`, PyTorch may generate Triton kernels through TorchInductor. A Python operation like:

```
def f(a):
    return torch.mean(a)
```

contains a reduction because mean is

$$\text{mean}(\mathbf{x}) = \frac{1}{N} \sum_{i=0}^{N-1} x_i.$$

TorchInductor can recognize the reduction and generate a Triton kernel with reduction-specific metadata and heuristics. Conceptually, the generated code contains something like:

```
tmp = tl.load(input_ptr + offsets, mask=mask, other=0.0)
partial = tl.sum(tmp, axis=0)
mean = partial / n
tl.store(output_ptr, mean)
```

The important point is not the exact syntax. The important point is that the compiler recognizes:

$$\text{mean} = \frac{\text{sum}}{\text{count}}.$$

Therefore, it must emit a sum reduction followed by a division.

8.5 Triton Reduction Hierarchy

Triton reductions also follow a hierarchy similar to CUDA reductions:

within program instance → within vector lanes → across program instances if needed.

At a lower level, compiler-generated reductions may use:

- per-thread or per-lane accumulation;
- warp-level reductions;
- shared memory for inter-warp reductions;
- block-level reductions;
- multiple kernel launches for large reductions.

This mirrors the CUDA hierarchy:

registers → warp → shared memory → global memory → kernel-level orchestration.

9 Common Misconceptions

9.1 Misconception 1: A Reduction Is Just a For Loop

A reduction can be written as a for loop, but a GPU reduction is a parallel cooperation problem. The difficult part is not the operator $+$, $\max$, or $\min$. The difficult part is mapping the operation to threads, memory, and synchronization.

9.2 Misconception 2: More Threads Always Means Faster

Launching many threads does not guarantee speed. If most threads are inactive because of divergence, the hardware is wasted. The thread mapping must preserve useful work per warp.

9.3 Misconception 3: Shared Memory Automatically Makes Kernels Faster

Shared memory helps only when it reduces expensive global memory traffic and is used with good access patterns. For tiny inputs, kernel launch overhead may dominate, so the measured runtime may not improve much.

9.4 Misconception 4: One Block Is Enough

A one-block reduction only works when the whole input fits into that block's processing capacity. Large reductions require multiple blocks, multiple passes, atomics, or hierarchical decomposition.

9.5 Misconception 5: GPU Reductions Are Always Deterministic

Parallel reductions can change the order of floating-point operations. Since floating-point addition is not associative, results can differ slightly from serial CPU results or from run to run when atomics are involved.

9.6 Misconception 6: FP16 Inputs Should Always Accumulate in FP16

Accumulating many FP16 values in FP16 can lose information. Many high-performance kernels use FP32 accumulators even when inputs and outputs are FP16 or BF16.

9.7 Misconception 7: Mean Is a Primitive Separate from Sum

Mean is usually implemented as a sum reduction followed by division:

$$\text{mean}(\mathbf{x}) = \frac{1}{N} \sum_i x_i.$$

This matters because mean inherits the numerical and performance properties of sum reduction.

10 Practical Takeaways

1. A reduction collapses many input values into fewer output values, often one scalar.
2. Reductions are everywhere in machine learning systems.
3. A serial reduction has span $O(N)$, while a tree reduction has span $O(\log N)$.
4. The total work remains $O(N)$, but the parallel critical path becomes much shorter.
5. The first naive CUDA reduction often suffers from severe control divergence.
6. A better stride strategy keeps active threads contiguous.
7. Shared memory avoids repeated global memory accesses for intermediate values.
8. One-block reductions are limited by block size and shared memory capacity.
9. Large inputs require segmented multi-block reduction.
10. Atomics are simple and correct but can bottleneck under contention.
11. Thread coarsening lets each thread reduce multiple values locally in registers.
12. Reductions are usually memory-bandwidth-bound because they have low arithmetic intensity.
13. Floating-point reduction order affects numerical results.
14. Production frameworks use generic reduction kernels, templates, code generation, and heuristics.
15. PyTorch eager CUDA kernels and TorchInductor-generated Triton kernels both rely on reduction-specific strategies.

11 Project or Experiment Ideas

11.1 Experiment 1: Naive Versus Contiguous-Thread Reduction

Implement two kernels:

1. one using the modulo-based active-thread condition;
2. one using the contiguous condition $t < \text{stride}$.

Profile both with Nsight Compute and compare:

- branch efficiency;
- warp execution efficiency;
- kernel duration;
- global memory throughput.

Expected result: the contiguous-thread version should show much better branch efficiency.

11.2 Experiment 2: Global Memory Versus Shared Memory

Implement a reduction that repeatedly uses global memory for intermediate results and another that uses shared memory.

Measure:

- L1 hit rate;
- global load transactions;
- shared memory throughput;
- total kernel duration.

Expected result: shared memory should reduce expensive global memory traffic, especially for larger block-level reductions.

11.3 Experiment 3: Atomic Versus Two-Pass Reduction

For a very large input, compare:

1. one kernel where each block uses `atomicAdd` into one output scalar;
2. two kernels where the first writes block partials and the second reduces partials.

Measure how performance changes as the number of blocks grows.

Expected result: atomics are simple but may become slower when many blocks contend for the same output address.

11.4 Experiment 4: Thread Coarsening Sweep

Implement a coarsened reduction and sweep the coarsening factor:

$$C \in \{1, 2, 4, 8, 16\}.$$

Measure:

- runtime;
- achieved occupancy;
- register usage;
- memory throughput.

Expected result: moderate coarsening can help, but excessive coarsening may hurt due to register pressure or reduced parallelism.

11.5 Experiment 5: FP16 Versus FP32 Accumulation

Create FP16 inputs and compare:

1. FP16 accumulation;
2. FP32 accumulation with FP16 output.

Use inputs with many small values and a few large values. Compare numerical error against a high-precision CPU reference.

Expected result: FP32 accumulation should be more accurate.

11.6 Experiment 6: PyTorch Reduction Inspection

Use `torch.compile` on a function such as:

```
def f(x):  
    return x.mean()
```

Inspect the generated code and identify:

- the reduction operation;
- the block size heuristic;
- whether Triton uses `tl.sum`;
- how the final division by N is represented.

11.7 Experiment 7: Implement Reductions in CUDA, Triton, and PyTorch

Implement sum reduction in three ways:

1. pure PyTorch using `torch.sum`;
2. custom Triton kernel;
3. custom CUDA extension.

Benchmark for different input sizes:

$$N \in \{1024, 10^5, 10^7, 10^8\}.$$

Compare performance and development complexity.

12 Final Mental Model

A reduction is a tree-shaped computation disguised as a simple loop. On a CPU, it is natural to think:

one accumulator, one element at a time.

On a GPU, the right mental model is:

many local partial reductions $\rightarrow$ block-level reduction $\rightarrow$ grid-level reduction $\rightarrow$ final scalar.

At each level of the hierarchy, the best storage and synchronization mechanism changes:

registers inside a thread,

warp-level cooperation inside a warp,

shared memory inside a block,

global memory or atomics across blocks.

The performance checklist for reductions is:

1. Keep active threads contiguous.

2. Avoid unnecessary branch divergence.
3. Use coalesced global loads.
4. Move intermediate values into shared memory or registers.
5. Synchronize only when necessary.
6. Use atomics carefully.
7. Coarsen threads when it improves useful work per thread.
8. Accumulate in a numerically appropriate dtype.
9. Profile with Nsight Compute rather than guessing.

The final engineering intuition is:

Reductions are simple mathematically but subtle architecturally. The operation is only one line, but the performance comes from choosing the right hierarchy: reduce within threads, then within warps, then within blocks, then across blocks, while respecting memory bandwidth, synchronization, divergence, and floating-point accuracy.